Diffusion Meets Options: Hierarchical Generative Skill Composition for Temporally-Extended Tasks

Zeyu Feng[1], Hao Luan[1], Kevin Yuchen Ma[1], and Harold Soh[1,2]

***Abstract*— Safe and successful deployment of robots requires not only the ability to generate complex plans but also the capacity to frequently replan and correct execution errors. This paper addresses the challenge of long-horizon trajectory planning under temporally extended objectives in a receding horizon manner. To this end, we propose DOPPLER, a data-driven hierarchical framework that generates and updates plans based on instruction specified by linear temporal logic (LTL). Our method decomposes temporal tasks into chain of options with hierarchical reinforcement learning from offline non-expert datasets. It leverages diffusion models to generate options with low-level actions. We devise a determinantal-guided posterior sampling technique during batch generation, which improves the speed and diversity of diffusion generated options, leading to more efficient querying. Experiments on robot navigation and manipulation tasks demonstrate that DOPPLER can generate sequences of trajectories that progressively satisfy the specified formulae for obstacle avoidance and sequential visitation.**

I. INTRODUCTION

As robots take on more complex real-world tasks, they must handle instructions that extend over time, such as "*put away the groceries before starting dinner*" or "*charge after finishing all cleaning tasks*." Formal languages like Linear Temporal Logic (LTL) provide a structured way to specify such tasks, enabling agents to reason about sequences of actions over time [1], [2]. However, planning under LTL constraints remains challenging, particularly in offline settings where agents must learn from fixed datasets without active exploration. Standard data-driven policy learning approaches struggle with the non-Markovian nature of LTL rewards and distribution shifts between the dataset and the deployment environment can degrade performance.

In this work, we propose **D**iffusion **O**ption **P**lanning by **P**rogressing **LTL**s for **E**ffective **R**eceding-horizon control (DOPPLER), an offline hierarchical reinforcement learning (RL) framework that generates receding-horizon trajectories to satisfy given LTL instructions. We propose diffusion-based options to generate *diverse* and *expressive* behaviors while ensuring that trajectories remain within the support of the offline dataset. To tackle the challenges of option selection and policy regularization in offline settings, we introduce a *diversity-guided sampling* approach that encourages exploration across different modes of the data distribution while avoiding out-of-distribution actions.

This research / project is supported by A*STAR under its National Robotics Programme (NRP) (Award M23NBK0053).

[1]Department of Computer Science, School of Computing, National University of Singapore, Singapore. {zeyu, haoluan, mayuchen, harold}@comp.nus.edu.sg

[2]Smart Systems Institute, National University of Singapore.

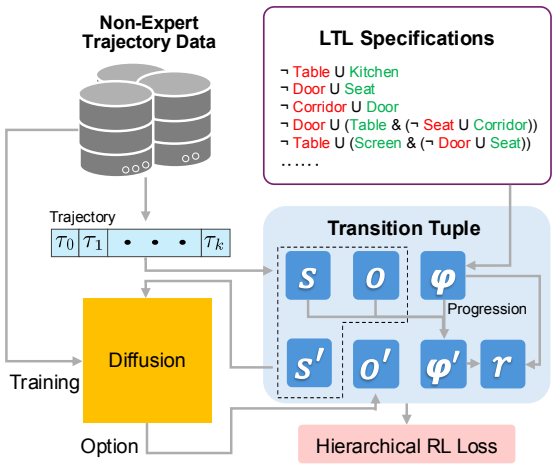


Fig. 1. Overview of DOPPLER framework. The model is trained using non-expert trajectory data and LTL specifications. Trajectories are sampled from the diffusion model to generate options, which are then used to form transition tuples that include the state (s), option (o), and LTL formula (φ). These tuples are processed with LTL progression to produce the next state (s'), next option (o'), updated LTL formula (φ'), and reward (r). The hierarchical RL loss is then computed to guide the learning process.

DOPPLER combines hierarchical RL with diffusion-based options, enabling *closed-loop* control under LTL constraints using only *offline non-expert* data. This is in contrast to prior work on trajectory planning under temporal constraints [3] and hierarchical diffusion frameworks for subgoal generation [4], [5], which either lacks closed-loop hierarchical planning capabilities or depends on expert-level demonstrations. Experiments show that DOPPLER achieves higher success rates than strong baselines on tasks with temporal specifications. In simulation, it effectively handles complex, long-horizon navigation tasks that prior methods struggle with. On a real quadruped robot, DOPPLER demonstrates robustness to control noise and external perturbations, successfully accomplishing LTL tasks where other approaches fail.

We believe DOPPLER represents a significant step toward learning agents capable of executing complex, temporally extended tasks. By leveraging diffusion-based options, DOPPLER improves generalization across diverse task variations while maintaining trajectory feasibility. The ability to learn from offline non-expert data reduces reliance on expert demonstrations, enabling broader applicability in real-world settings. To summarize, this paper makes three key contributions:

- A hierarchical RL approach designed for data-driven LTL planning with diffusion-based options;

- A diversity-guided sampling approach for diffusion models that enhances option discovery and policy regularization;
- Experimental results that validate the effectiveness of receding horizon planning for LTL tasks and demonstrate robustness to runtime deviations.

We hope our work lays the foundation for future research in integrating rich semantic temporal reasoning with data-driven policy learning.

II. Preliminaries

In this work, we aim to train an agent using an offline dataset to learn policies that generate trajectories satisfying Linear Temporal Logic (LTL) specifications. Our approach employs offline hierarchical reinforcement learning, where options are represented using diffusion models. In this section, we provide a succinct review of LTL, hierarchical reinforcement learning, and diffusion policies.

A. Formal Task Specification via LTL

To unambiguously define the task, we employ Linear Temporal Logic (LTL). LTL is a propositional modal logic with temporal modalities [6] that allows us to formally specify properties or events characterizing the successful completion of a task. These properties and events are drawn from a domain-specific finite set of atomic propositions $\mathcal{P}$. The set of LTL formulae Ψ is recursively defined in Backus-Naur form as follows [7], [8]:

$$\varphi := p \mid \neg\varphi \mid \varphi \wedge \psi \mid \bigcirc \varphi \mid \varphi \,\mathsf{U}\, \psi,$$

where $p \in \mathcal{P}$ and $\varphi, \psi \in \Psi$. $\neg$ (*negation*) and $\wedge$ (*and*) are Boolean operators while $\bigcirc$ (*next*) and U (*until*) are temporal operators. These basic operators can be used to derive other commonly-used operators, $\mathsf{true} = \varphi \vee \neg\varphi$, $\varphi \vee \psi = \neg(\neg\varphi \wedge \neg\psi)$ and $\Diamond\varphi = \mathsf{true}\,\mathsf{U}\,\varphi$ (*eventually* φ).

LTL formulae are evaluated over infinite sequences of truth assignments $\boldsymbol{\sigma} = \langle \sigma_0, \sigma_1, \sigma_2, \ldots \rangle$, where $\sigma_t \in \{0,1\}^{|\mathcal{P}|}$ and $\sigma_{t,p} = 1$ iff p is true at time step t. $\langle \boldsymbol{\sigma}, t \rangle \models \varphi$ denotes $\boldsymbol{\sigma}$ *satisfies* φ at time $t \geq 0$, which can be initially defined over atomic propositions and then extended to more logical operators according to their semantics. Given a σ, an LTL φ can be *progressed* to reflect which parts of φ have been satisfied and which remain to be achieved [1]. The one-step progression of φ on σ is defined as follows:

- $\mathrm{prog}(\sigma, p) = \mathsf{true}$ if $\sigma_p = 1$, where $p \in \mathcal{P}$
- $\mathrm{prog}(\sigma, \neg\varphi) = \neg\,\mathrm{prog}(\sigma, \varphi)$
- $\mathrm{prog}(\sigma, \varphi \wedge \psi) = \mathrm{prog}(\sigma, \varphi) \wedge \mathrm{prog}(\sigma, \psi)$
- $\mathrm{prog}(\sigma, \bigcirc\varphi) = \varphi$
- $\mathrm{prog}(\sigma, \varphi\,\mathsf{U}\,\psi) = \mathrm{prog}(\sigma, \psi) \vee (\mathrm{prog}(\sigma, \varphi) \wedge \varphi\,\mathsf{U}\,\psi)$

We also overload $\mathrm{prog}(\boldsymbol{\sigma}_{0:k}, \varphi)$ to indicate multistep progression over $\boldsymbol{\sigma}_{0:k}$. To facilitate effective LTL learning on a dataset of trajectories, we restrict our consideration to LTL formulas that can be satisfied or falsified within a finite number of steps. This category includes co-safe LTLs [9]–[11] and finite LTLs [12]. In this work, we focus on the former, which allows for evaluation over arbitrarily long sequences.

B. Hierarchical RL with options

We formulate RL within the framework of Markov decision processes (MDPs) [13]. An MDP $M = (\mathcal{S}, \mathcal{A}, p, r, \gamma)$ consists of state space $\mathcal{S}$, action space $\mathcal{A}$, transition probability function $p : \mathcal{S} \times \mathcal{A} \to \mathscr{P}(\mathcal{S})$[1], reward function $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ and discount factor $\gamma \in [0,1)$. Given a policy $\pi : \mathcal{S} \to \mathscr{P}(\mathcal{A})$ and an initial state $\mathbf{s}_0 \sim \mu_0 \in \mathscr{P}(\mathcal{S})$, an agent generates a trajectory $\boldsymbol{\tau} = (\mathbf{s}_t, \mathbf{a}_t)_{t=0}^{\infty}$ based on the distributions specified by π and p. The agent receives a discounted return $R(\boldsymbol{\tau}) = \sum_{t=0}^{\infty} \gamma^t r(\mathbf{s}_t, \mathbf{a}_t)$ along its trajectory and aims to maximize the expected return $\mathbb{E}_{\mu_0,\pi,p}[R(\boldsymbol{\tau})]$. The action-value function $Q^{\pi}(\mathbf{s}, \mathbf{a}) = \mathbb{E}_{\pi,p}[R(\boldsymbol{\tau}) \,|\mathbf{s}_0 = \mathbf{s}, \mathbf{a}_0 = \mathbf{a}]$ represents this expected return conditioned on a specific initial $\mathbf{s}$ and $\mathbf{a}$. In RL, p and r are unknown [14] to the learner.

The option-based framework incorporates temporally extended behaviors termed *options* into MDPs [15]. An option $o = \langle \mathcal{I}, \pi_o, \mathcal{B} \rangle$ is available to the agent in $\mathbf{s}$ iff $\mathbf{s}$ belongs to the initialization set $\mathcal{I} \subseteq \mathcal{S}$. Once the agent takes o, it follows π_o until o terminates, which occurs after t steps according to the probability $\mathcal{B} : \mathcal{S}^t \to [0,1]$. Policies are defined over both primitive actions $\mathcal{A}$ and options $\mathcal{O}$, *i.e.*, $\pi : \mathcal{S} \to \mathscr{P}(\mathcal{A} \cup \mathcal{O})$. The corresponding option-value function is $Q^{\pi}(\mathbf{s}, o) = \mathbb{E}_{o,\pi,p}[R(\boldsymbol{\tau}) \,|\mathcal{E}(o\pi, \mathbf{s})]$, where $\mathcal{E}(o\pi, \mathbf{s})$ denotes the event that o is initiated in $\mathbf{s}$ at $t = 0$, terminates at t with probability $\mathcal{B}(\mathbf{s}_1, \ldots, \mathbf{s}_t)$, and π is followed after o terminates.

C. Diffusion-based Policies

Existing diffusion-based policies directly generate a finite trajectory $\boldsymbol{\tau}_{0:k} = (\mathbf{s}_t, \mathbf{a}_t)_{t\in[k+1]}$[2] by training on a pre-collected dataset of trajectories. These policies can be trained using expert demonstrations for imitation learning [16], or with offline datasets guided by a Q function for policy improvement [17]. Formally, a step-dependent neural network s_θ is trained to approximate the score function $\nabla_{\boldsymbol{\tau}^i_{0:k}} \log p_i(\boldsymbol{\tau}^i_{0:k})$ with denoising score matching objective [18]:

$$\min_{\theta} \mathbb{E}_{i, \boldsymbol{\tau}^i_{0:k}, \boldsymbol{\tau}^0_{0:k}} \left[\left\| s_\theta(\boldsymbol{\tau}^i_{0:k}, i) - \nabla_{\boldsymbol{\tau}^i_{0:k}} \log p(\boldsymbol{\tau}^i_{0:k} | \boldsymbol{\tau}^0_{0:k}) \right\|^2 \right], \tag{1}$$

in which $\boldsymbol{\tau}^i_{0:k} \sim p(\boldsymbol{\tau}^i_{0:k} | \boldsymbol{\tau}^0_{0:k})$ is the data trajectory $\boldsymbol{\tau}^0_{0:k} \sim p_0(\boldsymbol{\tau}^0_{0:k})$ corrupted with noise by an N-step discrete approximation of forward diffusion process $p(\boldsymbol{\tau}^i_{0:k} | \boldsymbol{\tau}^{i-1}_{0:k})$ and $i \sim \mathcal{U}\{1, 2, \ldots, N\}$. For example, in Denoising Diffusion Probabilistic Models (DDPM) [19], $p(\boldsymbol{\tau}^i_{0:k} | \boldsymbol{\tau}^0_{0:k}) = \mathcal{N}(\sqrt{\bar{\alpha}_i}\boldsymbol{\tau}^0_{0:k}, (1 - \bar{\alpha}_i)\boldsymbol{I})$, $\bar{\alpha}_i := \prod_{j=1}^{i} \alpha_i$, $\alpha_i := 1 - \beta_i$ and $\{\beta_i\}$ is a sequence of positive noise scales $0 < \beta_1, \beta_2, \ldots, \beta_N < 1$. Diffusion models can generate plans conditioned on a start state by inpainting [17] or classifier-free guidance [20].

[1] $\mathscr{P}(\mathcal{S})$ defines the space of probability distributions over a set $\mathcal{S}$.

[2] The subscript $0:k$ indicates that a vector has $k+1$ elements, indexed by integers $0, 1, \ldots, k$. $[N]$ denotes the set of integers $\{0, 1, \ldots, N-1\}$.

III. METHOD: DOPPLER

In this section, we introduce DOPPLER, our offline hierarchical reinforcement learning framework for Linear Temporal Logic (LTL) instructions. At a high level, DOPPLER integrates hierarchical RL with diffusion models to effectively address the challenges of planning under LTL constraints in an offline setting (see Figure 1).

We begin by presenting our offline hierarchical RL approach for LTL instructions. This includes the construction of a product MDP to handle the non-Markovian nature of LTL rewards, the definition of options and rewards, and the learning of the option-value function from an offline dataset. Next, we describe how we represent options using diffusion models and introduce a novel diverse sampling technique. This approach allows us to obtain a rich set of options that are both expressive and within the support of the offline dataset, ensuring effective policy regularization.

A. Offline Hierarchical RL for LTL Instructions

Recall that we adopt an options framework for hierarchical reinforcement learning and our agent is tasked with generating behavior that satisfies a given Linear Temporal Logic (LTL) formula $\varphi \in \Psi$. The first challenge is that LTL satisfaction is evaluated over an entire trajectory. As such, defining a standard Markov Decision Process (MDP) M over the native states $\mathcal{S}$ is problematic since a non-Markovian reward function cannot be defined over single states.

To address this issue, we construct a product MDP M_Ψ such that the optimal Markov policies in M_Ψ recover the optimal policies for the non-Markovian reward function in M [21], [22]. More formally, we define $M_\Psi = (\mathcal{S}_\Psi, \mathcal{A}, p_\Psi, r_\Psi, \gamma)$, where $\mathcal{S}_\Psi = \mathcal{S} \times \Psi$. We assume that the properties or events related to a temporal task can be detected from the environmental state/action information and define a labeling function $L : \mathcal{S} \times \mathcal{A} \to \{0,1\}^{|\mathcal{P}|}$ that maps each state-action pair $(\mathbf{s}_t, \mathbf{a}_t)$ to a truth assignment $\sigma_t = L(\mathbf{s}_t, \mathbf{a}_t)$. We consider LTL progression [1], [21] as part of the transitions, and thus, the transition probability in M_Ψ is defined as $p_\Psi(\langle \mathbf{s}', \varphi' \rangle | \langle \mathbf{s}, \varphi \rangle, a) = p(\mathbf{s}'|\mathbf{s}, a)$ if $\varphi' = \text{prog}(L(\mathbf{s}, \mathbf{a}), \varphi)$ (and 0 otherwise).

Options and Rewards. We incorporate options into M_Ψ as described in Section II-B. We defer the detailed option definition to the next section and it suffices to assume that each option is a trajectory $o = \boldsymbol{\tau}_{0:k} = (\mathbf{a}_t, \mathbf{s}_{t+1})_{t=0}^{k-1}$. At state $\mathbf{s}_0$, the agent follows option o, which terminates after k steps. We assume that the options have an initiation set $\mathcal{I}$ covering the entire state space.

Next, we develop an option reward function such that the agent can either (i) accomplish the instructed LTL formula φ within the k steps of an option or (ii) transition to a state that enables completion of φ in the future. In our product MDP M_Ψ, the agent receives a reward at $(\mathbf{s}_t, \varphi)$ given by:

$$r_\Psi(\mathbf{s}_t, \varphi, \mathbf{a}_t) = \begin{cases} 1 & \text{if } \text{prog}(\sigma_t, \varphi) = \mathsf{true} \text{ and } \varphi \neq \mathsf{true} \\ -1 & \text{if } \text{prog}(\sigma_t, \varphi) = \mathsf{false} \text{ and } \varphi \neq \mathsf{false} \\ 0 & \text{otherwise.} \end{cases} \tag{2}$$

Algorithm 1 Offline Hierarchical RL for LTL objectives

Require: Datasets $\mathcal{D} = \{\boldsymbol{\tau}^{(d)}\}$ and $\mathcal{D}_\Psi$, Diffusion Model $\widehat{p}_\theta(o|\mathbf{s})$.
Initialize critics Q_{ϕ_1}, Q_{ϕ_2} and targets $Q_{\phi_1'}$, $Q_{\phi_2'}$
for $e = 0$ to E **do**
  Sample $\{\boldsymbol{\tau}^{(b)}\}_{b\in[B]}$ uniformly from $\mathcal{D}$
  Sample $\{\varphi^{(b)}\}_{b\in[B]}$ uniformly from $\text{cl}(\mathcal{D}_\Psi)$
  ▷ *Execute following in parallel for all* $b \in [B]$.
  Construct transition $\left(\mathbf{s}_0^{(b)}, \varphi^{(b)}, o^{(b)}, r_\Psi^{(b)}, \mathbf{s}_k^{(b)}, \varphi_k^{(b)}\right)$
  Propose M target options $o'^{(b)}_{(m)} \sim \widehat{p}_\theta\left(\cdot|\mathbf{s}_k^{(b)}\right)$
  Select best $o'^{(b)}$ by $Q_{\phi_1'}$
  Get clipped Gaussian noised version $\tilde{o}'^{(b)}$
  Get loss $\ell_j^{(b)}$ by Eq. (4) with double Q-learning
  Update $\phi_j \leftarrow \phi_j - \eta\nabla_{\phi_j}\frac{1}{B}\sum_b \ell_j^{(b)}$, $j \in \{1,2\}$
  if e mod e_0 **then**
    $\phi_j' = \lambda\phi_j + (1-\lambda)\phi_j'$, $j \in \{1,2\}$
  end if
end for
return Q_{ϕ_1}

where $\sigma_t = L(\mathbf{s}_t, \mathbf{a}_t)$. The option reward is then defined as $r_\Psi(\mathbf{s}_0, \varphi, o) = \sum_{t=0}^{k-1} \gamma^t r_t$, where $r_t = r_\Psi(\mathbf{s}_t, \varphi_t, \mathbf{a}_t)$ and $\varphi_t = \text{prog}(\boldsymbol{\sigma}_{0:t}, \varphi)$.

Learning an Option Critic. With the above definitions, we can now proceed to learn an option-value function using an offline dataset. A high-level summary of our learning algorithm is shown in Algorithm 1. According to the Bellman equation, the optimal option-value function has the form:

$$Q^*(\langle \mathbf{s}, \varphi \rangle, o) = r_\Psi(\mathbf{s}, \varphi, o) + \gamma^k \sum_{\mathbf{s}', \varphi'} p(\mathbf{s}', \varphi'|\mathbf{s}, o) \max_{o'} Q^*(\langle \mathbf{s}', \varphi' \rangle, o') \tag{3}$$

where $r_\Psi(\mathbf{s}, \varphi, o) = \mathbb{E}_{p_\Psi}\left[r_0 + \cdots + \gamma^{k-1} r_{k-1}\right]$ and $p(\mathbf{s}', \varphi'|\mathbf{s}, o)$ is the probability of transitioning to $\mathbf{s}'$ and φ' when executing option o under p_Ψ. We aim to obtain an option critic $Q(\langle \mathbf{s}, \varphi \rangle, o)$ that approximates Q^*. In our offline setting, we are provided with a dataset of *non-expert* trajectories $\mathcal{D}$ and a non-exhaustive dataset of LTL specifications $\mathcal{D}_\Psi$; in our experiments, we use tasks from prior work [3], [22].

We represent Q using a neural network that takes in state-option pairs and LTL formula embeddings. LTL embeddings can be obtained by using Graph Neural Networks (GNNs) that process graph representations of LTLs [23]–[25]. In this work, the LTL formula embedding is computed using a Relational Graph Convolutional Network (R-GCN) [26], which we found performs well on new LTL formulae with the same template structure seen during training.

We train Q by minimizing a temporal difference loss,

$$\left(r_\Psi(\mathbf{s}_0, \varphi, o) + \gamma^k \max_{o_k \in \mathcal{O}_{\mathbf{s}_k}} Q(\langle \mathbf{s}_k, \varphi_k \rangle, o_k) - Q(\langle \mathbf{s}_0, \varphi \rangle, o)\right)^2 \tag{4}$$

using transition tuples $(\mathbf{s}_0, \varphi, o, r_\Psi, \mathbf{s}_k, \varphi_k)$ constructed using the dataset, the option set $\mathcal{O}_{\mathbf{s}_k}$, and the training LTL space. Specifically, we sample a *short-horizon* trajectory $\boldsymbol{\tau}_{0:k} \sim p_0(\boldsymbol{\tau}_{0:k})$ from the dataset. Each $\boldsymbol{\tau}_{0:k} = (\mathbf{s}_t, \mathbf{a}_t)_{t=0}^k$ includes

current state $\mathbf{s}_0$, option $o = (\mathbf{a}_t, \mathbf{s}_{t+1})_{t=0}^{k-1}$ and next state $\mathbf{s}_k$. We use L to obtain a truth assignment sequence $\boldsymbol{\sigma}_{0:k}$ and compute the reward $r_\Psi(\mathbf{s}_0, \varphi, o) = \sum_{t=0}^{k-1} \gamma^t r_t$. The next LTL φ_k is obtained via LTL progression. We construct the training LTL space from the progression closure of formulas $\varphi \in \mathcal{D}_\Psi$, denoted $\mathrm{cl}(\mathcal{D}_\Psi)$. The loss function Eq. (4) is trained by uniformly sampling trajectories from dataset and LTL instructions from $\mathrm{cl}(\mathcal{D}_\Psi)$.

To stabilize the training, we leverage several popular techniques for Q networks. This includes using time delayed version of Q [27], clipped double Q-Learning [28], [29], and smoothing the target policy by injecting noise into the target actions [29], [30].

B. Options with Diffusion

Thus far, we have not yet fully defined our option set. Ideally, we would like a rich option set that is expressive enough to cover the diverse requirements of LTL instructions. However, one crucial issue is that in the offline setting, the dataset available to the learner is fixed and exploration is not permitted. As such, the options should generate trajectories within the support of the offline dataset to ensure reliable Q-value estimation.

To address this problem, we propose to regularize the policy space using diffusion options, i.e., we represent our option set $\mathcal{O}$ using a diffusion model and limit the policy's co-domain to be $\mathcal{O}$ (instead of $\mathcal{A} \cup \mathcal{O}$). Diffusion models form a rich policy class that can capture multimodal distributions, while generating samples within the training dataset.

Once a diffusion model $\widehat{p}_\theta(\boldsymbol{\tau}_{0:k}|\mathbf{s})$ is trained on the dataset, it can be used to sample options $o = \boldsymbol{\tau}_{0:k} \sim \widehat{p}_\theta(\boldsymbol{\tau}_{0:k}|\mathbf{s})$. However, during both training (Eq. 4) and deployment, we need to search over options to maximize the Q function. With diffusion models, finding the optimal option by sampling may require a large sample size to reduce approximation error. In addition, the samples may lack diversity, with many similar trajectories from a few modes of the distribution.

Q-Guidance. One potential way forward is to leverage the trained Q-network to perform conditional sampling. This form of classifier guidance can be achieved by methods such as posterior sampling. However, we find that in practice, Q-guided generation tends to produce low-quality trajectories (see Section V-B).

Diversity Sampling. We propose an alternative sampling approach that covers different modes of data distribution, while keeping the sample batch size M as small as possible.

The key idea is that each generated trajectory should be distinct from the others in the batch. A standard diffusion model generates options using an approximate conditional score $s_\theta(\cdot|\mathbf{s}_0) \approx \nabla_{\boldsymbol{\tau}^i_{0:k}} \log p_i\left(\boldsymbol{\tau}^i_{0:k}|\mathbf{s}_0\right)$. We instead propose to sample from a posterior that conditions on the $M-1$ other samples with the conditional score function $\nabla_{\boldsymbol{\tau}^i_{(1)}} \log p_i\left(\boldsymbol{\tau}^i_{(1)}|\left\{\hat{\boldsymbol{\tau}}_{(m)}\right\}_{m=2,\dots,M}\right)$. In the following text we omit the subscripts $0:k$ and conditioned state $\mathbf{s}_0$ for clarity.

With this change, the M samples are no longer independent. We leverage Bayes' rule and sample estimation [31] to express the conditional score as the sum of two terms: $\nabla_{\boldsymbol{\tau}^i_{(1)}} \log p_i\left(\boldsymbol{\tau}^i_{(1)}\right)$ and $\nabla_{\boldsymbol{\tau}^i_{(1)}} \log\, p\big(\left\{\hat{\boldsymbol{\tau}}_{(m)}\right\}_{m=2,\dots,M}|\hat{\boldsymbol{\tau}}_{(1)}\big)$, where the noiseless trajectory $\hat{\boldsymbol{\tau}}_{(1)}$ is estimated via Tweedie's formula [32] $\hat{\boldsymbol{\tau}}_{(1)} = \frac{1}{\sqrt{\bar{\alpha}_i}}\big(\boldsymbol{\tau}^i_{(1)} + (1-\bar{\alpha}_i)\nabla_{\boldsymbol{\tau}^i_{(1)}} \log p_i\big(\boldsymbol{\tau}^i_{(1)}\big)\big)$ and $p_i\big(\{\hat{\boldsymbol{\tau}}_{(m)}\}_{m=2,\dots,M} \mid \boldsymbol{\tau}^i_{(1)}\big)$ is approximated by point estimation $p\big(\{\hat{\boldsymbol{\tau}}_{(m)}\}_{m=2,\dots,M} \mid \hat{\boldsymbol{\tau}}_{(1)}\big)$.

We model this conditional probability using a differentiable probabilistic model. To encourage sample diversity, the conditional probability should be high if $\hat{\boldsymbol{\tau}}_{(1)}, \dots, \hat{\boldsymbol{\tau}}_{(M)}$ are pairwise dissimilar, and low if any of them are similar. Thus, the unnormalized probability of the generated set can be modeled by a similarity matrix analogous to determinantal point processes [33],

$$p\left(\hat{\boldsymbol{\tau}}_{(1)}|\left\{\hat{\boldsymbol{\tau}}_{(m)}\right\}_{m=2}^{M}\right) = p\left(\hat{\boldsymbol{\tau}}_{(1)}\right)\det\left(L_M\right)/Z, \quad (5)$$

where L_M is the similarity matrix with elements indexed by integers (u, v), $1 \le u, v \le M$ measuring the similarity (*e.g.*, cosine similarity) between $\hat{\boldsymbol{\tau}}_{(u)}$ and $\hat{\boldsymbol{\tau}}_{(v)}$, and $Z = \int_{\hat{\boldsymbol{\tau}}_{(1)}} p\left(\hat{\boldsymbol{\tau}}_{(1)}\right)\det\left(L_M\right)\mathrm{d}\hat{\boldsymbol{\tau}}_{(1)}$ is a normalizing constant [34]. Since the normalizing constant Z does not depend on $\hat{\boldsymbol{\tau}}_{(1)}$, the gradient of the point estimate becomes

$$\nabla_{\boldsymbol{\tau}^i_{(1)}} \log p\left(\left\{\hat{\boldsymbol{\tau}}_{(m)}\right\}_{m=2}^{M}|\hat{\boldsymbol{\tau}}_{(1)}\right) = \nabla_{\boldsymbol{\tau}^i_{(1)}} \log\det\left(L_M\right). \quad (6)$$

This gradient can be plugged into the reverse process for posterior sampling as summarized in Algorithm 2. Our approach maximizes the determinant of the similarity matrix, biasing the generated trajectories to be distinct from one another, thus promoting diversity.

Algorithm 2 Diversity Guided Batch Sampling

Require: N, s_θ, M, $\{\zeta_i\}_{i=1}^N$
▷ *Execute following in parallel for all* $m \in [M]$.
$\boldsymbol{\tau}^N_{(m)} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, $m \in [M]$
for $i = N-1$ **to** 0 **do**
  $\hat{s}^{(m)} \leftarrow s_\theta\left(\boldsymbol{\tau}^i_{(m)}, i\right)$
  Predict $\hat{\boldsymbol{\tau}}^0_{(m)}$ via Tweedie's formula
  Reverse one step to compute $\boldsymbol{\tau}^{i-1}_{(m)}$
  $L_{u,v} = \cos\left(\hat{\boldsymbol{\tau}}^0_{(u)}, \hat{\boldsymbol{\tau}}^0_{(v)}\right)$, $u, v \in [M]$
  $\boldsymbol{\tau}^{i-1}_{(m)} \leftarrow \boldsymbol{\tau}^{i-1}_{(m)} + \zeta_i \nabla_{\boldsymbol{\tau}^i_{(m)}} \log\det\left(L\right)$
end for
return $\left\{\hat{\boldsymbol{\tau}}^0_{(m)}\right\}_{m\in[M]}$

IV. Related Work

LTL is widely-used to specify high-level, temporally extended requirements in robot tasks [1], [2], [35]. Many existing data-driven methods for LTL planning learn in an *online* fashion [21], [22], [36]–[38]. These methods can be effective but may also lead to unsafe interactions during the trial-and-error learning process.

Here, we focus on offline learning that does not require interactions with the environment. Existing offline methods typically rely on knowledge of the environmental dynamics, are limited to certain types of LTL specifications [39], [40],

TABLE I

SATISFACTION RATE (%) (↑) ON DIFFERENT LTLS IN MAZE2D.

Env.	Method	Training LTL$_f$s		Testing LTL$_f$s	
		Planning	Rollout	Planning	Rollout
Medium	DIFFUSER	15.0±0.7	13.4±0.6	11.6±1.4	10.1±1.2
	LTLDOG-S	77.9±5.7	31.8±2.6	68.4±6.7	28.7±3.5
	LTLDOG-R	51.8±1.8	39.5±1.6	43.3±4.4	30.6±1.9
	DOPPLER	**95.3±0.8**	**90.2±3.3**	**95.7±0.6**	**89.6±2.1**
Large	DIFFUSER	13.5±0.4	12.8±0.1	11.6±2.3	11.5±1.7
	LTLDOG-S	73.8±2.4	32.6±1.4	66.6±2.7	24.9±1.7
	LTLDOG-R	66.9±0.6	47.4±0.8	57.5±2.3	39.0±2.9
	DOPPLER	**95.4±0.3**	**92.0±0.5**	**94.1±1.6**	**89.8±2.0**

TABLE II

PERFORMANCE ON DIFFERENT LTLS IN PUSHT

Method\Performance	LTL Set	Satisf. rate (%) ↑	Score ↑
DIFFUSION POLICY	Training	21.8±1.02	0.388±0.006
	Test	32.8±1.48	0.385±0.028
LTLDOG-S	Training	28.7±1.30	0.294±0.006
	Test	38.9±2.91	0.300±0.023
LTLDOG-R	Training	71.9±1.77	0.289±0.001
	Test	62.9±1.72	0.353±0.004
DOPPLER	Training	**87.4±0.84**	**0.386±0.022**
	Test	**94.1±1.85**	**0.432±0.042**

or require events to be explicitly labeled in the dataset [41]. The closest related work is our previous model LTLDOG [3], which uses posterior sampling with diffusion models and LTL model checkers to generate plans under LTL instructions. However, LTLDOG is restricted to non-hierarchical open-loop planning. Subgoal generation via hierarchical diffusion frameworks has been explored using methods such as graph search [4] and keyframe discovery [5]. However, these methods are not designed for satisfying LTL specifications. To our knowledge, we are the first to successfully combine offline hierarchical RL with diffusion-based options for LTL planning, enabling effective closed-loop control under complex temporal specifications. DOPPLER learns *without* access to task-oriented expert trajectories and as experiments will show, can produce effective long-horizon trajectories to comply with novel LTL instructions.

V. EXPERIMENTS

Our experiments aim to evaluate DOPPLER's ability to learn and plan for temporal tasks that are not explicitly demonstrated in the dataset. First, we compare DOPPLER to state-of-the-art data-driven methods in simulated benchmark environments. Then, we demonstrate DOPPLER's robustness in dealing with control noise and external perturbations in real-world tasks through a case study on a quadruped robot (Figure 4). The section concludes with an ablation study of our diversity sampling approach.

A. Experimental Setup

Environments. We conduct experiments in benchmark simulation environments —Maze2d [17], [42] and PushT [16] — and on a real quadruped robot. In Maze2d (Fig. 2), tasks specified by LTL formulae require complex, long-horizon navigation beyond simple goal-reaching. LTL properties and events are determined by the agent's presence in key regions of the maze. We adopt the LTL setting in prior work [3], where the regions of states and actions for each atomic proposition in $\mathcal{P}$ are disjoint. In the PushT task (Fig. 3), the agent manipulates a `T` block via a controllable mover to accomplish temporal instructions. In real-world experiments, we test LTL tasks on a quadruped robot in indoor environments (Fig. 4). To increase problem difficulty, we perturbed the robot by moving it manually to a different position during the rollout (reminiscent of the kidnapped robot problem). This was to test if the planner could recover from large deviations from the intended plan.

TABLE III

RESULTS FOR REAL-WORLD NAVIGATION TASKS.

Environment	Method\Performance	Satisfaction rate (%) ↑	
		LTL	Perturbation
Lab	LTLDOG	70	45
	DOPPLER	**95**	**95**
Office	LTLDOG	30	15
	DOPPLER	**95**	**85**

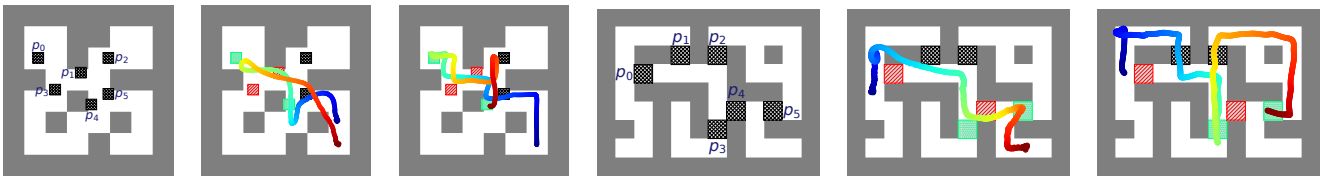
Compared Methods. We compare DOPPLER against other data-driven planning methods: DIFFUSER and DIFFUSION POLICY, which are diffusion-based methods for sampling plans but lack the ability to plan over temporal constraints. The most relevant work is LTLDOG [3], which generates full trajectories under LTL instructions. We evaluate performance using the average LTL satisfaction rate during planning and rollout, averaged over 10 random start locations for each LTL. In the real-robot experiment, we executed 80 trajectories in total for each method.

B. Results

Comparative Analysis of Methods. Tables I and II present the performance under randomly-generated LTL specifications using the `Until` sampler, as in [3], [22] (200 for Maze2d and 36 for PushT), which contain different sequences of regions to visit and avoid. We observe that DOPPLER achieves substantially higher success rates than all baselines. These results demonstrate that DOPPLER effectively discovers compositions of skills and benefits from closed-loop planning.

Examples in Figure 2 show that DOPPLER's closed-loop nature successfully handles long trajectories that exceed the fixed horizon on which LTLDOG was trained. Note that some tasks may be impossible to accomplish due to physical limitations such as the agent's starting location, obstacles, or the arrangement of propositional regions in the maze, which limits performance below 100%.

Real-World Study on Quadruped Robot. As shown in Table III and Figure 5, DOPPLER achieves significantly higher satisfaction rates compared to LTLDOG. LTLDOG lacks replanning capabilities and was prone to task violations when perturbed to be close to avoidance regions. In contrast, DOPPLER exhibits more robust behavior, with only a few failure cases; these failures typically occurred in



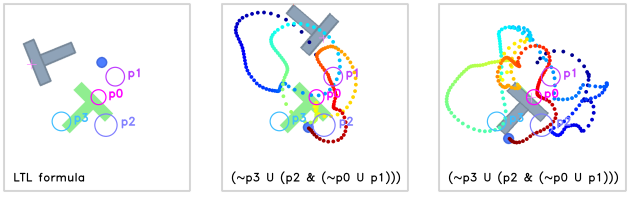
(a) Medium (b) LTLDoG (c) DOPPLER (d) Large (e) LTLDoG (f) DOPPLER

Fig. 2. Setup and sample trajectories for Maze2D environments. (a) and (d) depict Maze2D Medium and Large, each containing six non-overlapping regions (hatched squares labeled with p_x) used to evaluate atomic propositions in $\mathcal{P}$. The agent is tasked with visiting specific regions in different temporally extended sequences. (b) and (e) illustrate the trajectories generated by LTLDoG and our proposed method (DOPPLER) (from blue to red) under the specification $\varphi = \neg p_3 \,\mathsf{U}\, (p_0 \wedge (\neg p_1 \,\mathsf{U}\, p_4))$. (c) and (f) show trajectories generated under the specification $\varphi = \neg p_0 \,\mathsf{U}\, (p_3 \wedge (\neg p_4 \,\mathsf{U}\, p_5))$. Our method (DOPPLER) successfully satisfies these complex LTL specifications by avoiding regions with $\neg$ propositions (red zones) before reaching the designated green regions, as demonstrated in panels (e) and (f).

TABLE IV

ABLATION STUDY ON DIFFERENT LTLS IN MAZE2D-LARGE.

Method\Performance	Training LTLs						Testing LTLs					
	Satisf. rate (%) ↑		Failure rate (%) ↓		Successful Steps ↓		Satisf. rate (%) ↑		Failure rate (%) ↓		Successful Steps ↓	
	Planning	Rollout	Planning	Rollout	Planning	Rollout	Planning	Rollout	Planning	Rollout	Planning	Rollout
Q-Guidance	82.4±1.0	80.1±0.7	2.6±0.5	**2.6±0.3**	759.4±13.7	819.4±4.0	79.1±1.2	76.1±0.9	2.8±0.5	2.6±0.9	814.9±10.8	905.5±15.5
Ours (w.o. diversity)	91.0±0.8	91.0±0.7	4.6±2.3	5.6±2.1	515.1±13.8	572.5±14.2	89.5±0.7	88.4±2.2	4.0±2.7	5.4±3.2	587.9±8.0	647.3±13.1
Ours (DOPPLER)	**95.4±0.3**	**92.0±0.5**	**2.2±0.4**	2.8±0.6	**386.9±5.2**	**445.7±12.4**	**94.1±1.6**	**89.8±2.0**	**1.8±0.7**	**2.4±0.6**	**429.1±10.0**	**516.7±14.5**

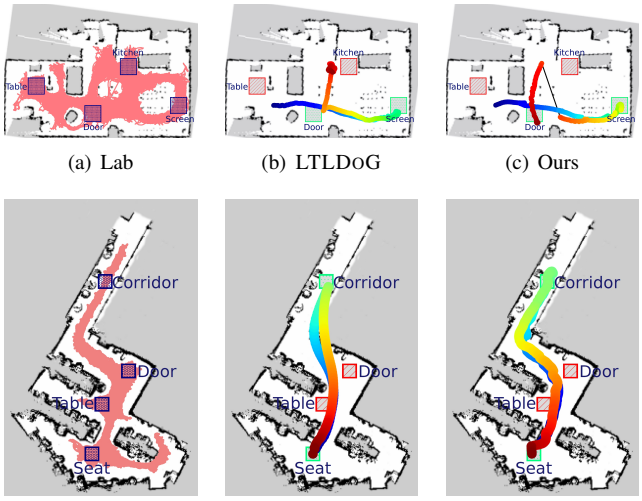


(a) PushT Setup (b) LTLDoG (c) DOPPLER (Ours)

Fig. 3. The PushT manipulation environment. (a) A robot arm's end effector (circles filled in blue) should manipulate the T block (gray) to a goal pose (green) *and* visit some regions (hollow circles marked with p_x) under different temporally-extended orders before completion. (b) LTLDoG-R does not comply with the LTL nor completes the manipulation. (c) In contrast, DOPPLER can satisfy the LTL *and* complete the manipulation task.

(a) Studio-like Lab (b) Office room (c) Unitree Go2

Fig. 4. Real world environments for quadruped robot navigation.



(a) Lab (b) LTLDoG (c) Ours

(d) Office (e) LTLDoG (f) Ours

Fig. 5. Results in real-world rooms. Each room has 4 key locations ((a) and (d)). The instructed LTL is $\neg$Table U (Screen $\wedge$ ($\neg$Kitchen U Door)) for lab (first row) and $\neg$Door U (Corridor $\wedge$ ($\neg$Table U Seat)) for office (second row). LTLDoG is unable to recover from perturbations (b) and cannot generate a valid plan (e), while ours ((c) and (f)) can achieve both.

areas outside the data distribution, where the diffusion model struggled to generate valid options.

Ablation Study on the Diversity Sampling. Our proposed diversity sampling method significantly outperforms standard sampling and Q-guidance (see TABLE IV). The gap between Q-guidance and the other methods is relatively large, which we posit is due to mismatched Q-guidance gradients with the denoising updates in diffusion.

VI. CONCLUSION, LIMITATIONS AND FUTURE WORK

In this paper, we introduced DOPPLER, an offline hierarchical reinforcement learning framework that leverages diffusion-based options to satisfy complex LTL instructions. Our approach effectively integrates diffusion models into a hierarchical RL setting, enabling the agent to generate diverse and expressive behaviors while adhering to the constraints of the offline dataset. Experiments in both simulation and real-world environments demonstrated that DOPPLER significantly outperforms state-of-the-art methods in satisfying temporal specifications and exhibits robustness to control noise and external perturbations.

Our work opens up several avenues for future research, particularly in handling cases where relevant behavioral fragments are missing from the offline dataset. Since DOPPLER relies on available trajectory fragments to construct feasible options, its performance may be limited when key behaviors are absent. An interesting approach may be to combine offline learning with limited online adaptation.

REFERENCES

[1] F. Bacchus and F. Kabanza, "Using temporal logics to express search control knowledge for planning," *Artif. Intell.*, vol. 116, no. 1, pp. 123–191, 2000.

[2] G. Fainekos, H. Kress-Gazit, and G. Pappas, "Temporal logic motion planning for mobile robots," in *IEEE Int. Conf. Robot. Automat.*, 2005, pp. 2020–2025.

[3] Z. Feng, H. Luan, P. Goyal, and H. Soh, "LTLDoG: Satisfying temporally-extended symbolic constraints for safe diffusion-based planning," *IEEE Robotics and Automation Letters*, vol. 9, no. 10, pp. 8571–8578, 2024.

[4] W. Li, X. Wang, B. Jin, and H. Zha, "Hierarchical diffusion for offline decision making," in *Int. Conf. Mach. Learn.*, ser. Proceedings of Machine Learning Research, A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, Eds., vol. 202. PMLR, 23–29 Jul 2023, pp. 20 035–20 064.

[5] X. Ma, S. Patidar, I. Haughton, and S. James, "Hierarchical diffusion policy for kinematics-aware multi-task robotic manipulation," *IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024.

[6] A. Pnueli, "The temporal logic of programs," in *18th Annu. Symp. Found. Comput. Sci.*, 1977, pp. 46–57.

[7] C. Baier and J. Katoen, *Principles of Model Checking*. MIT Press, 2008.

[8] C. Belta, B. Yordanov, and E. A. Gol, *Formal Methods for Discrete-Time Dynamical Systems*. Springer, 2017, vol. 89.

[9] B. Lacerda, D. Parker, and N. Hawes, "Optimal and dynamic planning for markov decision processes with co-safe ltl specifications," in *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2014, pp. 1511–1516.

[10] ——, "Optimal policy generation for partially satisfiable co-safe LTL specifications," in *Int. Joint Conf. Artif. Intell.*, vol. 15, July 2015, pp. 1587–1593.

[11] O. Kupferman and M. Y. Vardi, "Model checking of safety properties," *Formal methods in system design*, vol. 19, pp. 291–314, 2001.

[12] A. Camacho, J. Baier, C. Muise, and S. McIlraith, "Finite LTL synthesis as planning," in *Proc. Int. Conf. Automated Planning and Scheduling*, vol. 28, 2018, pp. 29–38.

[13] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*, 1st ed. USA: John Wiley & Sons, Inc., 1994.

[14] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.

[15] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," *Artificial Intelligence*, vol. 112, no. 1, pp. 181–211, 1999.

[16] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," in *Proc. Robot.: Sci. and Syst. (RSS)*, 2023.

[17] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine, "Planning with diffusion for flexible behavior synthesis," in *Int. Conf. Mach. Learn.*, vol. 162, 2022, pp. 9902–9915.

[18] P. Vincent, "A connection between score matching and denoising autoencoders," *Neural Comput.*, vol. 23, no. 7, pp. 1661–1674, 2011.

[19] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Inf. Process. Syst.*, 2020, pp. 6840–6851.

[20] J. Ho and T. Salimans, "Classifier-free diffusion guidance," in *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*, 2021.

[21] R. Toro Icarte, T. Q. Klassen, R. Valenzano, and S. A. McIlraith, "Teaching multiple tasks to an RL agent using LTL," in *Proc. Int. Conf. Autonomous Agents Multiagent Syst.*, 2018, pp. 452–461.

[22] P. Vaezipoor, A. C. Li, R. A. T. Icarte, and S. A. Mcilraith, "LTL2Action: Generalizing LTL instructions for multi-task RL," in *Int. Conf. Mach. Learn.*, 2021, pp. 10 497–10 508.

[23] M. Gori, G. Monfardini, and F. Scarselli, "A new model for learning in graph domains," in *Proc. IEEE Int. Joint Conf. Neural Netw.*, vol. 2, 2005, pp. 729–734.

[24] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Networks*, vol. 20, no. 1, pp. 61–80, 2009.

[25] Y. Xie, F. Zhou, and H. Soh, "Embedding symbolic temporal knowledge into deep sequential models," in *IEEE Int. Conf. Robot. Automat.*, 2021, pp. 4267–4273.

[26] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. Van Den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *The Semantic Web*, 2018, pp. 593–607.

[27] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *Int. Conf. Mach. Learn.*, ser. Proceedings of Machine Learning Research, M. F. Balcan and K. Q. Weinberger, Eds., vol. 48. New York, New York, USA: PMLR, 20–22 Jun 2016, pp. 1928–1937.

[28] H. Hasselt, "Double q-learning," in *Advances in Neural Inf. Process. Syst.*, J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel, and A. Culotta, Eds., vol. 23. Curran Associates, Inc., 2010.

[29] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Int. Conf. Mach. Learn.*, ser. Proceedings of Machine Learning Research, J. Dy and A. Krause, Eds., vol. 80. PMLR, 10–15 Jul 2018, pp. 1587–1596.

[30] R. Simmons-Edler, B. Eisner, E. Mitchell, S. Seung, and D. Lee, "Q-learning for continuous actions with cross-entropy guided policies," 2019.

[31] H. Chung, J. Kim, M. T. Mccann, M. L. Klasky, and J. C. Ye, "Diffusion posterior sampling for general noisy inverse problems," in *Int. Conf. Learn. Representations*, 2023.

[32] B. Efron, "Tweedie's formula and selection bias," *J. Amer. Statistical Assoc.*, vol. 106, no. 496, pp. 1602–1614, 2011.

[33] A. Kulesza, B. Taskar, *et al.*, "Determinantal point processes for machine learning," *Foundations and Trends® in Machine Learning*, vol. 5, no. 2–3, pp. 123–286, 2012.

[34] J. Song, Q. Zhang, H. Yin, M. Mardani, M.-Y. Liu, J. Kautz, Y. Chen, and A. Vahdat, "Loss-guided diffusion models for plug-and-play controllable generation," in *Int. Conf. Mach. Learn.*, vol. 202, 2023, pp. 32 483–32 498.

[35] J. A. Baier and S. A. McIlraith, "Planning with temporally extended goals using heuristic search," in *Proc. Int. Conf. Automated Planning and Scheduling*, 2006, p. 342–345.

[36] C. Yang, M. L. Littman, and M. Carbin, "On the (in)tractability of reinforcement learning for LTL objectives," in *Int. Joint Conf. Artif. Intell.*, 2022, pp. 3650–3658.

[37] C. Voloshin, H. M. Le, S. Chaudhuri, and Y. Yue, "Policy optimization with linear temporal logic constraints," in *Advances in Neural Inf. Process. Syst.*, 2022, pp. 17 690–17 702.

[38] D. Tian, H. Fang, Q. Yang, H. Yu, W. Liang, and Y. Wu, "Reinforcement learning under temporal logic constraints as a sequence modeling problem," *Robotics and Autonomous Systems*, vol. 161, p. 104351, 2023.

[39] T. Wongpiromsarn, U. Topcu, and R. M. Murray, "Receding horizon temporal logic planning for dynamical systems," in *Proceedings of the 48h IEEE Conference on Decision and Control (CDC) held jointly with 2009 28th Chinese Control Conference*, 2009, pp. 5997–6004.

[40] J. X. Liu, A. Shah, E. Rosen, M. Jia, G. Konidaris, and S. Tellex, "Skill transfer for temporal task specification," in *IEEE Int. Conf. Robot. Automat.*, 2024, pp. 2535–2541.

[41] Y. Wang, N. Figueroa, S. Li, A. Shah, and J. Shah, "Temporal logic imitation: Learning plan-satisficing motion policies from demonstrations," in *6th Annual Conference on Robot Learning*, 2022.

[42] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, "D4RL: Datasets for deep data-driven reinforcement learning," *arXiv preprint arXiv:2004.07219*, 2020.